

Índice

1. Consideraciones	4
2. Introducción	4
3. Arquitectura	5
3.1. Comunicacion entre vista y componentes	5
3.2. Flujo de implementación de nuevos componentes	6
3.3. Componentes multi-hilo	7
3.4. Comunicacion de eventos	8
4. SDP	9
4.1. Introducción y Rol en la Arquitectura	9
4.2. Abstracción y Diseño	9
4.3. Estructura del mensaje SDP	9
4.3.1. Nivel de Sesión	9
4.3.2. Nivel de Media	10
4.4. Negociación y Fujo Offer/Answer	10
4.4.1. Generación de Oferta	10
4.4.2. Generacion y Procesamiento de Respuesta	10
4.5. Serialización y Parseo	11
4.6. Persistencia	11
5. Sesión RTP	12
5.1. Comunicación RTP	12
5.1.1. Implementacion	12
5.1.2. Jitter Buffer	12
5.2. Comunicación RTCP	13
6. Comunicación Cliente - Servidor	14
6.1. Protocolo PCA	14
6.2. Encriptación de mensajes	14
7. Servidor	17
7.1. Funcion del servidor	17
7.1.1. Persistencia de usuarios	17
7.1.2. Limitacion de usuarios conectados	17
7.1.3. Gestion de usuarios	17
7.1.4. Gestion interaccion entre usuarios	17
7.2. Estructura	18
8. Seguridad	20
8.1. DTLS	20

8.1.1.	Crate UDP-DTLS	20
8.1.2.	Rol de DTLS - Flujo de aplicación	20
8.1.3.	Generación de material criptográfico con OpenSSL	21
8.1.3.1.	Generación y almacenamiento del certificado X.509	21
8.1.3.2.	Fingerprint SHA-256	21
8.1.4.	Negociación de roles DTLS	21
8.1.4.1.	Análisis del atributo setup en SDP	21
8.1.4.2.	Determinación de rol local	21
8.1.5.	Handshake DTLS - Flujo	22
8.1.6.	Extracción de claves SRTP	22
8.1.6.1.	Derivación de claves	22
8.1.6.2.	Estructura de claves	22
8.1.6.3.	Preparación del contexto SRTP	23
8.2.	SRTP	23
8.2.1.	Rol de SRTP - Flujo de aplicación	23
8.2.2.	Arquitectura del módulo	23
8.2.2.1.	Contextos de cifrado y decifrado	24
8.2.2.2.	Integración con threads RTP existentes	24
8.2.3.	Claves y perfiles	25
8.2.3.1.	Uso de las claves derivadas por DTLS	25
8.2.3.2.	Perfil SRTP	25
8.2.3.3.	Salt y master keys	25
8.2.4.	Cifrado de RTP	26
8.2.4.1.	Comparación de estructura de paquete RTP	26
8.2.4.2.	Proceso de cifrado	26
8.2.4.3.	Replay y autenticación - ROC	26
8.2.5.	Decifrado de RTP	26
8.2.5.1.	Validaciones	26
8.2.5.2.	Decodificación	27
8.2.5.3.	Verificación de autenticidad	27
9.	Conclusión	28
10.	Anexo - Captura y transmisión de audio	29
10.1.	Captura del audio	29
10.2.	Transmisión de audio	30
11.	Anexo II - Envío y recepción de archivos	31
11.1.	Protocolos	31
11.1.1.	SCTP	31
11.1.1.1.	Loop de procesamiento y sincronización	31
11.1.2.	DCEP	32

11.2. Demultiplexado	33
11.2.1. Identificación y separación	33
11.3. Arquitectura multithread	34
11.3.1. Multi-threading	34

1. Consideraciones

Las implementaciones de SDP, ICE, RTP/RTCP, DTLS y SRTP se desarrollan utilizando únicamente crates permitidos y abstraídos de manera mínima, con el objetivo de mantener la compatibilidad conceptual con WebRTC sin depender de librerías de alto nivel. Cada módulo del sistema se encuentra desacoplado, documentado y sujeto a pruebas unitarias e integración, siguiendo las buenas prácticas recomendadas por la cátedra.

Finalmente, la arquitectura prioriza la claridad, la seguridad y la eficiencia: se evita el uso de `unsafe`, se respetan los límites de concurrencia mediante canales y `Arc<Mutex>`, y se utilizan archivos de configuración y logs según los lineamientos del proyecto. Debido a la naturaleza en tiempo real de la aplicación, se contemplan también aspectos como la latencia, el ordenamiento de paquetes y el manejo de jitter en la reproducción del video.

2. Introducción

El presente proyecto consiste en construir una implementación de los componentes esenciales del stack WebRTC, utilizando exclusivamente el lenguaje Rust. El enfoque se centra en permitir la comunicación en tiempo real entre dos usuarios, incluyendo el intercambio de descripciones SDP, la negociación de conectividad mediante ICE, y la transmisión de video utilizando RTP y RTCP. Para garantizar la seguridad extremo a extremo, la solución incorpora DTLS para la autenticación entre pares y SRTP para el cifrado de los flujos de video, además de TLS para el cifrado de comunicación en servidor.

El objetivo final es obtener una plataforma funcional, extensible y eficiente, capaz de integrarse con un servidor central encargado del manejo de usuarios, signaling y estados de presencia.

3. Arquitectura

Antes de ahondar en cuestiones técnicas e implementativas sobre lo desarrollado en nuestro trabajo, primero introduciremos la arquitectura elegida para organizar los diferentes componentes de la aplicación.

El desarrollo de las funcionalidades solicitadas para la aplicación requirió que varios componentes con diferentes funciones deban comunicarse entre sí. Inicialmente, las funcionalidades no explicitaban necesariamente que componentes debían existir y que comunicación debía haber entre ellos; el panorama inicial era un conjunto de requerimientos desordenados, con pocas o ningunas especificaciones sobre cómo debía ser la implementación. Por ello, fue crucial definir cuál sería la estructura de nuestra implementación. Siempre que fue posible, estas decisiones se tomaron no de forma arbitraria, sino siguiendo heurísticas ya existentes para lidiar con esta situación tan común en el desarrollo de *software*.

Se hizo uso de muchas de las herramientas proporcionadas por Rust para que el desarrollo sea lo más similar al permitido por un lenguaje puramente orientado a objetos. Como se mencionará más adelante, se intentó desarrollar la mayor cantidad de funcionalidad posible usando *Test Driven Development*; para esto fue muy útil tener 'objetos polimórficos' (logrados gracias a los *traits* de Rust), sobre todo para simular el comportamiento de algunos componentes a la hora de testear otro componente. Además, las herramientas brindadas para asociar 'métodos' a los *structs* creados permitieron acercar el desarrollo a las ideas del paradigma de objetos. Por esto, en lo que resta de esta sección nos referiremos por momentos a 'objetos' o a 'polimorfismo', aunque no sean las palabras más adecuadas.

En la siguiente sección se detallará una de las primeras decisiones que debieron tomarse sobre la arquitectura del programa: la comunicación entre los componentes y la interfaz gráfica

3.1. Comunicación entre vista y componentes

Dado que el trabajo a realizar era -entre otras cosas- implementar una aplicación de video-llamadas, una de las primeras cosas que se afrontaron durante el desarrollo fue el despliegue de una interfaz gráfica con funcionalidades mínimas, a la que progresivamente se le podrían agregar funcionalidades. Una vez logrado esto, el siguiente paso fue implementar funcionalidades que respondan a las solicitudes de la interfaz gráfica. Y fue en ese punto que surgió uno de los primeros problemas sobre la arquitectura del software que se estaba desarrollando.

El principal problema que se debió resolver fue encontrar una manera de comunicar la interfaz gráfica con los componentes de la aplicación, de manera tal que el acoplamiento entre estas dos piezas sea, de ser posible, inexistente. Incluir lógica de una de estas secciones en la otra contraía problemas difíciles de resolver, que se acrecentaban a medida que la aplicación crecía. Si bien la necesidad de esta división suele conocerse, muchas veces ese conocimiento es más bien dogmático porque no se conocen los problemas reales que se originan al no respetar esta 'regla', y mucho menos se sabe cómo solucionarlos. Inicialmente, la falta de una estructura definida nos causó estos problemas, y nos permitió al menos reconocer cuáles eran los problemas a los que se debía prestar atención:

- Si incluíamos lógica relacionada a la interfaz en los componentes internos de la aplicación, como por ejemplo métodos para obtener su estado y luego mostrarlo, entonces la vista debía consultar por el estado de cada uno de los componentes individualmente, y así decidir qué hacer. En este caso, cualquier mínimo cambio en un componente individual se propagaba hasta la vista.
- Por otro lado, el problema era aún más grave y claro si incluíamos lógica de los componentes en la interfaz. Cualquier mínimo cambio en estos se propagaba a la interfaz generando grandes cambios, e incluso la responsabilidad de coordinar estos componentes quedaba relegada a la interfaz.

Para solucionar estos problemas se adoptó una arquitectura propuesta por los desarrolladores de *SmallTalk*. El artículo (Deacon 2009) consultado sobre su implementación propone una división de la aplicación en las siguientes partes:

- El modelo del dominio del problema o *Domain Model*, que engloba aquellos 'objetos' que cumplen con las funcionalidades requeridas para nuestro programa de forma aislada: iniciar sesión, llamar a un usuario, responder a una llamada, capturar video, enviar video, etc. La propuesta es que estos componentes no sepan absolutamente nada sobre las vistas, y que únicamente cumplan con un subconjunto de las funcionalidades que estén relacionados de forma muy cercana. En nuestra aplicación, se pueden mencionar al Comunicador; encargado de la comunicación de mensajes del servidor y escucharlos, la Recepción; encargada del registro e inicio de sesión usando un Comunicador, el Teléfono; encargado de enviar llamadas y recibirlas, etc.
- El modelo de la Aplicación o *Application Model*, que se presenta como un 'orquestador de los componentes internos de la Aplicación'. La idea de esta parte consiste en que se quiere resolver es una aplicación que permita realizar determinadas funciones, y por lo tanto la aplicación como tal puede considerarse parte del modelo de nuestra aplicación. Modelar esto permite tener un 'objeto' que se encargue de delegar las funcionalidades requeridas a los componentes que deban cumplirla, y además coordinar sus acciones. Por otro lado, por el contrario al modelo del dominio del problema, nuestra aplicación conoce que habrá 'observadores' que querrán conocer su estado para mostrarlo. Aun así, no conocerá a las vistas, sino que informará de la ocurrencia de 'eventos' a observadores genéricos, que mostrarán a su antojo los eventos de la aplicación.
- El controlador, que es mencionado como un componente opcional en las implementaciones de esta estructura. En nuestro caso, no vimos necesaria la creación de esta pieza.
- Las vistas, cuya función debería poder resumirse a 'escuchar eventos y mostrarlos'. En nuestro caso, exigimos que cualquier 'objeto' que pretenda actuar de Vista debe tener un método asociado a cada evento que pueda informarnos a la Aplicación. De esta manera, todos los eventos deberían ser informados al usuario, indistintamente de la Vista que se esté usando.

La estructura adoptada nos permitió implementar progresivamente las funcionalidades sin que el crecimiento en el tamaño de la aplicación afecte demasiado a las funcionalidades ya implementadas.

Si bien *Rust* no es un lenguaje orientado a objetos, las herramientas brindadas permitieron llegar a una estructura que puede representarse adecuadamente en un diagrama de clases. En la Figura 1 se puede ver el resultado final del mismo.

3.2. Flujo de implementación de nuevos componentes

Como se dijo antes, se utilizó la técnica de *Test Driven Development* para la implementación de nuevas funcionalidades a la aplicación. Con la estructura ya implementada, se pudo llegar a un procedimiento a seguir que se repitió para la mayoría de funcionalidades que se implementaron.

Dada una nueva funcionalidad a implementar, y luego de especificar que era lo mínimo que se necesitaba para considerar la funcionalidad como 'terminada', los pasos que se siguieron en la mayoría de casos fueron los siguientes:

1. Abarcar una porción de la funcionalidad en cada test de cada componente, hasta que el componente cumpla con su funcionalidad entera. Para este paso, en muchos casos se necesitaba comprobar que el componente se comunique de forma adecuada con otros componentes. Para eso se utilizaron 'objetos' simuladores, polimórficos a los 'objetos' reales. En *Rust*, pudimos lograr esto implementando un mismo *trait* tanto en el objeto real como en el objeto simulador.
2. Abarcar la funcionalidad en la Aplicación, usando 'objetos' simuladores para los componentes usados y para la vista

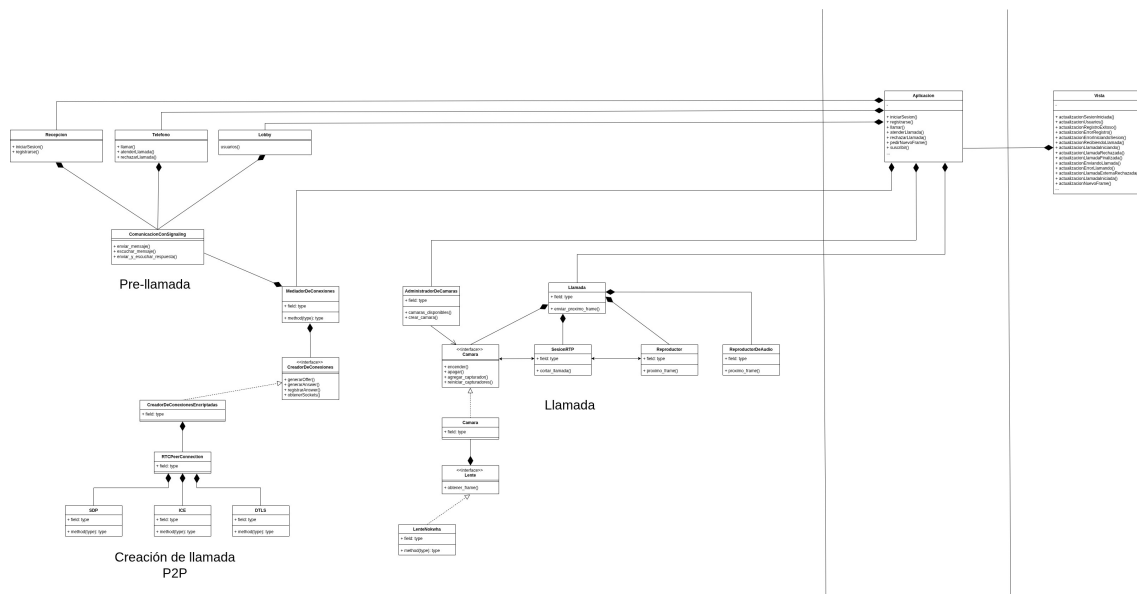


Figura 1: Diagrama de clases de la aplicación

3. Mostrar en la vista lo que fuera necesario al recibir los nuevos eventos generados

Se puede ver que el proceso consiste en dividir el flujo en secciones, y luego probar cada parte asumiendo que el resto de las partes funcionan bien (esto ultimo logrado con objetos simuladores). Haciendo un analisis -un poco rebuscado- de lo recién mencionado, se podría considerar que el algoritmo usado para añadir funcionalidades es un algoritmo *Greedy*: por cada sección buscamos el optimo haciendo pruebas y asumiendo que el resto de las cosas funcionaban bien, y como resultado final esperamos que la totalidad de la funcionalidad este correctamente implementada. Esto es una simple observación que parece fuera de lugar, pero permite entender el caracter iterativo que se buscó para la implementación de nuevas funcionalidades en la aplicación.

El uso de 'objetos' simuladores tambien permitio crear tests de integraci3n que comprueben que se realicen de forma correcta flujos enteros de funcionalidad en la aplicacion. Para esto basto con usar objetos simuladores de los dispositivos de entrada y salida usados en el programa (no se podia asumir que quien ejecute los tests iba a tener un microfono o una camara, por ejemplo). A modo de ejemplo, uno de los tests de integraci3n implementados comprueba que dos clientes logran registrarse e iniciar sesi3n en un mismo servidor, y que luego de llamarse y atender ambos entran a la pantalla de llamada.

3.3. Componentes multi-hilo

Dadas las características de nuestra aplicación, muchos de los componentes que la conformaban requerían de múltiples hilos de ejecución para realizar sus tareas. Durante el desarrollo, manejar esos hilos supuso un gran desafío a afrontar.

Pongamos de ejemplo al Telefono: una de sus funcionalidades es llamar a otros usuarios, y otra de ellas es escuchar si un usuario esta enviandonnos una llamada. Para la primera de ellas, el funcionamiento es relativamente sencillo: si al telefono se le indica que llame a un usuario, solo debe comprobar si esta en un estado valido (por ejemplo, no puede estar en una llamada) y dado el caso enviar un mensaje al Comunicador, que sera quien se comuniquen con el servidor. La segunda funcionalidad, sin embargo, tiene un funcionamiento diferente: el Telefono debera estar esperando a recibir una llamada para ejecutar las acciones adecuadas. La unica forma de resolver esta segunda tarea es tener un hilo de ejecución aparte, que este bloqueado hasta que una llamada llega. Asi, se puede ver que para la implementación del Telefono fue necesario mas de un hilo de ejecución.

A todos los ejemplos de componentes que requerían múltiples hilos para funcionar se los trató siguiendo una misma estrategia, que será explicada a continuación. En adelante, nos referiremos a estos componentes como 'componentes multi-hilo'.

Uno de los objetivos buscados en los componentes multi-hilo era que los demás componentes no tuvieran que conocer si el componente tenía uno o varios hilos, ni tampoco a cuál de esos hilos le estaba enviando un mensaje. Para lograr esto, fue necesario que los hilos de un mismo componente puedan comunicarse entre sí. La solución propuesta fue que los objetos sean 'dueños' de los hilos que poseían. Así, si una operación debe resolverse en otro de los hilos del componente, este debe informárselo. Esta comunicación se logra de forma natural usando una gran herramienta propuesta por *Rust*: los *channels*. De esta manera, la comunicación con los hilos de todos los objetos se realiza enviando instrucciones (normalmente variantes de un *enum*) mediante un *channel*. Cuando una operación debe delegarse a otro hilo, basta con enviarle una instrucción.

Otro de los problemas que se debieron resolver en los componentes multi-hilo fue el control del tiempo de vida de los hilos. Se puede ver que este problema se solucionó fácilmente gracias al funcionamiento logrado para el objetivo anterior: si cada componente es 'dueño' de sus hilos, entonces este será quien controle cuánto viven. Así, los componentes pueden cerrar sus hilos fácilmente enviándoles instrucciones.

3.4. Comunicación de eventos

Como se comentó al detallar la arquitectura elegida, la Aplicación comunica a sus observadores los eventos que ocurren en los componentes. Para lograr esto, hubo que separar a los eventos en dos categorías.

Por un lado están aquellos eventos generados al ejecutar operaciones sincrónicas en los componentes. Un ejemplo: cuando se le solicita a la aplicación que inicie sesión con unas credenciales determinadas, esta le delega el inicio de sesión a la Recepción. Luego de eso, dependiendo del resultado del inicio de sesión, deberá informarse un evento u otro. En estos casos, se eligió que sea la Aplicación quien informe estos eventos según el resultado informado por el componente.

Por otro lado, hay otro subconjunto de los eventos que se generan a partir de acciones que se generan desde el exterior, y no a partir del envío de un mensaje a la Aplicación. Por ejemplo, al recibir una llamada se genera un evento, pero ese evento no se produce por haber solicitado algo a la Aplicación. En nuestro programa, se decidió que este tipo de eventos sean comunicados por el mismo componente. Este funcionamiento fue sencillo de implementar gracias a los *channels* provistos por *Rust*: bastó con enviar a cada componente una 'punta de envío' de eventos, de forma tal que estos también lleguen a los observadores.

Para esta explicación se obviaron algunos detalles implementativos, como la existencia de un hilo 'notificador' que permita que nuevos observadores se suscriban para escuchar eventos de nuestra aplicación. Estos detalles están mejor explicados en la documentación del programa, y fueron analizados usando contenido multimedia en la presentación de este trabajo.

4. SDP

4.1. Introducción y Rol en la Arquitectura

El Protocolo de Descripción de Sesión (SDP) no actúa como un protocolo de transporte de medios en sí mismo, sino como un formato estándar para describir los parámetros de inicialización de una sesión multimedia. En el contexto de nuestra implementación, el SDP es el medio que permite realizar el intercambio de capacidades entre los peers, lo que les permitirá saber de antemano (antes de siquiera intentar conectarse), si sería posible establecer una conexión entre ambos, o no.

4.2. Abstracción y Diseño

Para obtener un sistema flexible, se optó por un diseño basado en *Traits*. Esta decisión desacopla la representación en memoria de las estructuras de datos de su representación textual (acorde al estándar propuesto en RFC).

Se definieron dos traits principales que rigen el comportamiento de cualquier componente SDP:

- **ParseableSdp**: Define la capacidad de construir una estructura interna a partir de un conjunto de líneas de texto crudo. Este trait impone que la función de parseo devuelva un **Result**, obligando al manejo explícito de errores ante formatos SDP malformados o incompletos.
- **SerializableSdp**: Estandariza la conversión inversa, permitiendo que las estructuras internas se serialicen al formato de texto plano con terminaciones requeridas por el estándar.

Esta separación nos permite asegurar que cualquier nuevo componente agregado al protocolo (como futuros atributos, o secciones completas que representen nuevas estructuras) puedan cumplir con la interfaz de serialización esperada.

4.3. Estructura del mensaje SDP

La implementación propuesta representa un mensaje SDP completo mediante la estructura **DescripcionDeSesion**. Esta estructura actúa como el contenedor de alto nivel que agrupa dos secciones lógicamente distintas: la información global de la sesión con los datos generales del peer, y las descripciones de los medios individuales.

La estructura interna se compone de:

- **sesion**: Una instancia única de **SesionSdp**, que contiene los metadatos globales, y representa el "header" del mensaje, la cabecera.
- **media**: Un vector de estructuras **DescripcionDeMedia**, permitiendo manejar múltiples flujos dentro de una misma negociación. En el mensaje SDP, las medias se representan como un bloque de datos aislado, delimitado por las líneas que empiezan por "m=" (o sea, desde la primera línea con m=, hasta el final del mensaje o hasta el inicio de la próxima media), donde se detallan las capacidades específicas de un determinado flujo de media.

4.3.1. Nivel de Sesión

El nivel de sesión, representado por **SesionSdp**, encapsula los parámetros globales. Esta estructura es responsable de manejar líneas críticas como:

- **v=** (versión): Fija en 0 para SDP.
- **o=** (origen): Incluye la gestión dinámica del identificador único de la sesión (Session ID) y la versión de la sesión, junto con la resolución de la dirección IP local del host.

- **c=** (conexión): Especifica la dirección base de la conexión, generalmente `IN IP4 IP_LOCAL`.
- **t=** (tiempo): Fijo en 0 0 para sesiones en tiempo real.

4.3.2. Nivel de Media

La estructura `DescripcionDeMedia` representa la complejidad real de la negociación. Cada bloque de media (iniciado por **m=**) define el tipo de contenido (audio/video), el puerto de transporte, el protocolo (RTP/AVP), y el formato de payload.

Nuestra implementación es capaz de gestionar listas dinámicas de atributos (**a=**), diferenciando entre:

- **Atributos de configuración:** Como el mapeo de tipos de payload (**a=rtpmap**).
- **Candidatos ICE:** La estructura mantiene vectores separados para candidatos locales y remotos (**a=candidate**), facilitando la lógica de conexión posterior sin "contaminar" la lista de atributos generales.

4.4. Negociación y Flujo Offer/Answer

La implementación soporta el modelo estándar de Oferta/Respuesta (Offer/Answer), adaptado para priorizar la transmisión de video en entornos controlados.

4.4.1. Generación de Oferta

Para iniciar una llamada, el sistema genera una `DescripcionDeSesion` configurada como *Offer*. En esta etapa, la implementación construye proactivamente una sección de media de tipo video. Basándose en la configuración local (`ConfigRoomRTC`), se instancia un bloque de media que declara el puerto RTP local y establece los parámetros predeterminados para el códec de video.

Específicamente, se estandarizó el uso del códec *H264* como el formato de codificación preferido. La función `crear_media_videoh264` automatiza la inclusión de los atributos **rtpmap** necesarios, simplificando el proceso de creación de ofertas válidas y reduciendo el margen de error en la negociación de códecs.

La elección de estandarizar un sólo códec (H264) responde al objetivo de la primera entrega: asegurar el envío y recepción de video funcional, utilizando un único crate a elección para la codificación/decodificación del mismo. Sin embargo, para mayor extensibilidad, dado que la estructura `DescripcionDeSesion` gestiona un vector de descripciones de media, y el proceso de intersección de capacidades se realiza a nivel de los identificadores de payload, el sistema está preparado para incorporar y negociar múltiples códecs y tipos de media en lo que a SDP se refiere. El mecanismo actual de intersección, encapsulado en funciones como `obtener_codecs_comunes`, se concentra en la lógica de búsqueda de intersección entre listas de capacidades, lo que minimiza el esfuerzo de refactorización cuando se introduzca soporte para nuevos códecs, o diferentes tipos de media.

4.4.2. Generación y Procesamiento de Respuesta

La generación de la respuesta (*Answer*) es un proceso crítico donde se determina la viabilidad de la comunicación. Al recibir una oferta remota, nuestro sistema no la acepta ciegamente, sino que realiza un análisis de compatibilidad mediante la función `generar_answer_desde_offer`.

El flujo de decisión es el siguiente:

1. **Clonación y Preparación:** Se iteran las secciones de media de la oferta recibida.
2. **Intersección de Capacidades:** Se verifica si existe una intersección no vacía entre los códecs ofrecidos por el par remoto y los soportados localmente.

3. **Filtrado de Atributos:** Si hay una coincidencia, la respuesta generada retiene únicamente las descripciones de media que contengan códecs que ambos peers soporten, eliminando aquellos que alguno de los dos no pueda decodificar. Esto asegura que el SDP resultante contenga un subconjunto válido de negociación.
4. **Rechazo de Media:** En caso de que un flujo no sea soportado (por ejemplo, un tipo de media desconocido o falta de códecs comunes), se ejecuta `rechazar_media`, que establece el puerto del bloque de media a 0, señal estándar en SDP para indicar que el flujo o la media han sido rechazados o deshabilitados, evitando que los peers intenten conectarse utilizando los datos de una media propuesta por el otro peer, que no soportan.

4.5. Serialización y Parseo

Al implementar `SerializableSdp`, tanto la sesión como las descripciones de media ordenan y formatean sus atributos antes de concatenarlos en un string final. Se presta especial atención a la correcta generación de líneas `a=candidate` y a la inclusión de la línea `a=mid` para la identificación de flujos.

Para el parseo, el sistema utiliza un enfoque que divide el texto entre líneas y detecta los límites de las secciones de media (cada nueva línea `m=` marca el inicio de un nuevo bloque). Esto permite manejar múltiples descripciones de media en un sólo mensaje, devolviendo vectores tipados y validando la presencia de campos obligatorios.

4.6. Persistencia

Como soporte (y para poder verificar que la generación de mensajes SDPs sea la esperada, considerando que con la presencia de un *Signaling Server* dichos mensajes pasan totalmente desapercibidos por el usuario), la estructura `DescripcionDeSesion` puede ser guardada en un archivo .sdp gracias a la función `guardar_en_archivo`.

5. Sesión RTP

Una vez establecida la conexión peer-to-peer a través de ICE y finalizado el intercambio de offer y answer SDP, los peers ya cuentan con lo suficiente como para iniciar la transmisión segura mediante DTLS/SRTP (lo cual será explicado más adelante en el informe).

A partir de este punto comienza la fase de captura, codificación, envío, recepción y decodificación del video.

Nuestro sistema implementa el protocolo RTP para la transmisión de media y RTCP para el control de la sesión como tal. Nuestro diseño contempla la eficiencia, tolerancia de pérdida y la correcta sincronización de los paquetes.

Al iniciar la sesión RTP se establecen las estructuras que se compartirán en RTP y RTCP.

5.1. Comunicación RTP

El protocolo **RTP (Real Time Transport Protocol)** es un protocolo diseñado para la transmisión de datos multimedia en tiempo real, como lo es en nuestro caso el video.

Estos datos multimedia se transportan a través del Socket UDP dentro de Paquetes RTP. Para nuestra implementación los campos que más nos interesan son el número de secuencia, el timestamp, el ssrc y el payload. El payload es la información que nos interesa transportar como tal.

5.1.1. Implementación

- Uno que se encarga de escuchar los datagramas ya codificados y listos para ser enviados al otro peer dentro de un paquete RTP. Este además actualiza las estadísticas de tipo **Sender** las cuales son importantes para luego utilizarlas dentro del protocolo RTCP.
- Y otro que recibe esos paquetes por el socket UDP y le envía esa información al **Jitter Buffer** del cual se hablará más adelante. También actualiza las estadísticas del **Receiver** por la misma razón.

5.1.2. Jitter Buffer

Para mitigar el manejo de pérdidas de paquetes y su reordenamiento se implementó un **trait Gestion Jitter Buffer** el cual internamente posee al Jitter Buffer en cuestión. Para la implementación del Jitter Buffer utilizamos un *árbol B* ya que por sus características y tiempos de ejecución nos pareció la mejor alternativa. Como se mencionó anteriormente la información de los paquetes RTP son transferidas a esta estructura.

Para la implementación del **trait Gestion Jitter Buffer** se optó por utilizar dos hilos:

- Uno el cual recibe el payload junto con el timestamp y los inserta dentro del árbol el cual los ordenará según el timestamp. Este hilo además mantiene información sobre cuál fue el último timestamp enviado, entonces cada vez que llega uno nuevo si este tiene un timestamp menor al último enviado se lo descarta ya que se lo considera como un paquete tardío.

Además cuenta con una cantidad máxima de timestamps junto a datagramas almacenados.

Se utiliza esta metodología tanto para la gestión del jitter buffer de audio como para la del video.

- El otro hilo cada cierto tiempo periódico se encarga de agarrar el payload con el timestamp más pequeño para así enviárselo al destinatario, que es el decodificador en el caso del video y el reproductor para el audio.

A continuación se explicará como se maneja cada uno y qué metodología se utilizó para la sincronización de estas dos medias.

Para el audio simplemente se hace lo dicho anteriormente y cada vez que se envía al reproductor audio se le actualiza al jitter buffer de video cual es su timestamp.

Respecto al video, como optimización y debido a la estructura del decoder se optó por no enviarle un datagrama nuevo si este se encuentra procesando otro, esto previene el delay ya que si se le enviara se generaría una especie de *cola* en la que se acumularían datagramas antiguos a largo plazo.

Además, antes de enviar un datagrama al decodificador, se verifica que el timestamp del datagrama no difiera del timestamp del último fragmento de audio enviado en más de 33ms. En caso de que esto suceda, se descarta el datagrama ya que significaría que en algún momento por algún problema externo hubo un retraso en el envío de paquetes de video.

5.2. Comunicación RTCP

El protocolo **RTCP (Real Time Control Protocol)** complementa al protocolo RTP, encargándose de monitorear la calidad de la sesión y realizar estadísticas respecto a esta.

En nuestra implementación RTCP opera enviando SR(Sender Reports) , RR(Receiver Reports) y BYE.

El paquete BYE juega un rol fundamental en nuestra implementación ya que cuando un peer decide finalizar la llamada envía este paquete mediante RTCP y la llamada finaliza para ambos peers.

Luego los paquetes SR y RR se utilizan para estadísticas.

6. Comunicación Cliente - Servidor

Teniendo las bases del protocolo RTP ya implementadas, nuestro programa permite visualizar otros usuarios con los cuales, potencialmente, se podría querer realizar una llamada. Para eso, en primer lugar, se debe llegar a un protocolo que permita que las partes puedan entenderse y coordinarse para lograr su cometido. Luego, una vez definido un protocolo, surge el desafío de lograr que los mensajes que se envíen por este medio no puedan ser interceptados e interpretados con malas intenciones. El desarrollo de estos problemas y las soluciones encontradas se detallarán en las siguientes secciones.

6.1. Protocolo PCA

Inicialmente, la tarea planteada fue encontrar un protocolo que logre satisfacer todas las necesidades de comunicación de nuestro programa, y que al mismo tiempo sea minimal. Luego de varias iteraciones, se concluyó que, en caso de ser necesario, sería de utilidad tener mensajes que aporten información y eviten que las partes deban comunicarse, evitando así mantener un estado interno sobre el historial de mensajes recibidos desde el exterior.

El conjunto de mensajes elegidos para desarrollar la comunicación fue el siguiente:

REGISTRAR usuario contraseña

REGISTRADO

ENTRAR usuario contraseña

LLAMAR usuario_destino

LLAMANDO usuario_origen

RECHAZO

OFFER offer

PEDIR OFFER

ANSWER answer

USUARIOS usuario_1;(DISP|OCUP)

CORTAR

SALIR

SALIO

ERROR mensaje

OK

6.2. Encriptación de mensajes

Para encriptar los mensajes entre el cliente y servidor se debieron analizar las problemáticas relacionadas a la encriptación entre dos partes, cuyo canal de comunicación no es seguro en ningún momento. Esta característica resulta fundamental a la hora de elegir un método de encriptación,

ya que la red proporciona un medio de comunicación que, aunque veloz, es altamente inseguro; sin demasiada complejidad, cualquiera puede actuar interrumpiendo los mensajes en su viaje por la red, ya sea para vulnerarlos o simplemente para robar información. Esto es lo que se conoce vulgarmente como un *man-in-the-middle attack*.

La primera de las problemáticas a afrontar fue la siguiente: si un intermediario lograra interceptar algún mensaje entre dos partes, se pretende que los datos que se obtengan sean inútiles, y que el intermediario sea incapaz de descifrar el significado real de esos mensajes. Este es un problema que se puede resolver de diversas maneras. En particular, por las características de nuestro medio de comunicación, lo óptimo resultó en elegir un método de **cifrado asimétrico**: antes de iniciar la comunicación, cada parte genera un par de claves; una pública y una privada. Luego, se realiza un intercambio de las claves públicas entre las partes. El funcionamiento de este tipo de encriptación radica en que cualquiera con una clave pública puede encriptar un mensaje, pero solo quien tenga la clave privada puede desencriptarlo. En nuestro caso, la herramienta nos permite que la otra parte encripte mensajes y los envíe, sin miedo a que sean descifrados. Si el mensaje es interceptado, su contenido no podrá leerse sin tener la clave privada correspondiente, que solo la tendrá quien la generó (esta última no es intercambiada bajo ninguna circunstancia).

En nuestro programa, la generación de claves públicas y privadas se realiza con una dependencia externa, que cuenta con una implementación del algoritmo RSA. Resumidamente, el funcionamiento de este algoritmo recae sobre la idea de que, dado un número determinado, factorizarlo como un producto de números primos es un problema NP-Completo, y por lo tanto, difícil (o imposible) de resolver sin calcular absolutamente todas las posibilidades.

Aunque pareciera que con esto el problema de encriptación de mensajes ya queda resuelto, todavía queda un problema muy importante por resolver. Supongamos que dos partes A y B quieren intercambiarse un mensaje utilizando el método de encriptación antes detallado. El método propone que, inicialmente, A envíe su clave pública y escuche la clave enviada por B. De la misma forma, B envíe su clave pública y reciba la clave pública de su contraparte. Si se mira con especial atención el flujo recién detallado, sale a la luz un nuevo problema a resolver: en el momento exacto en que A o B envían sus claves públicas, un intermediario C podría interrumpir la conexión y obtener la clave pública. En adelante, podría enviar cualquier mensaje que quisiera encriptándolo con esa clave, y la otra parte no podría saber jamás que ese mensaje no es, en realidad, de quien esperaba. Esto podría llevar ocasionar graves problemas ya que, por ejemplo, el servidor podría recibir mensajes e interpretar que esos mensajes fueron enviados por un usuario, cuando en realidad el contenido de ese mensaje podría estar alterado de forma maliciosa. Para resolver ese problema se diseñó una nueva capa sobre el flujo de mensajes de encriptación que se detalló antes. Aprovechando que luego de enviar las claves públicas los mensajes ya se envían encriptados, A generará una clave determinada, se la guardará y se la enviará a su contraparte B, encriptándola antes con su clave pública. La parte B, por su lado, realizará el mismo procedimiento. En adelante, antes de enviar un mensaje, la parte A deberá seguir el siguiente procedimiento (que es equivalente para B):

1. Realizar una copia del mensaje a enviar
2. Adjuntar la clave recibida por B al final del nuevo mensaje
3. *Hashear* el mensaje resultante, y guardarse el *hash* obtenido
4. Adjuntar el *hash* al final del mensaje.
5. Encriptar el mensaje resultante con la clave pública recibida, y enviarlo

Siguiendo este procedimiento, cuando B reciba el mensaje podrá desencriptarlo y comprobar su origen de la siguiente manera:

1. Se recibe el mensaje, que tiene un *hash* al final. Si tal *hash* no existe (deberá estar marcado su inicio con un carácter especial), se descarta el mensaje.
2. Se obtiene el mensaje original (sin el *hash*)

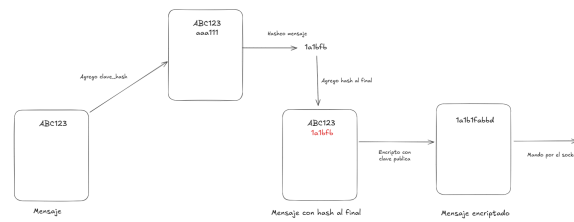


Figura 2: Encriptación del mensaje con clave hash

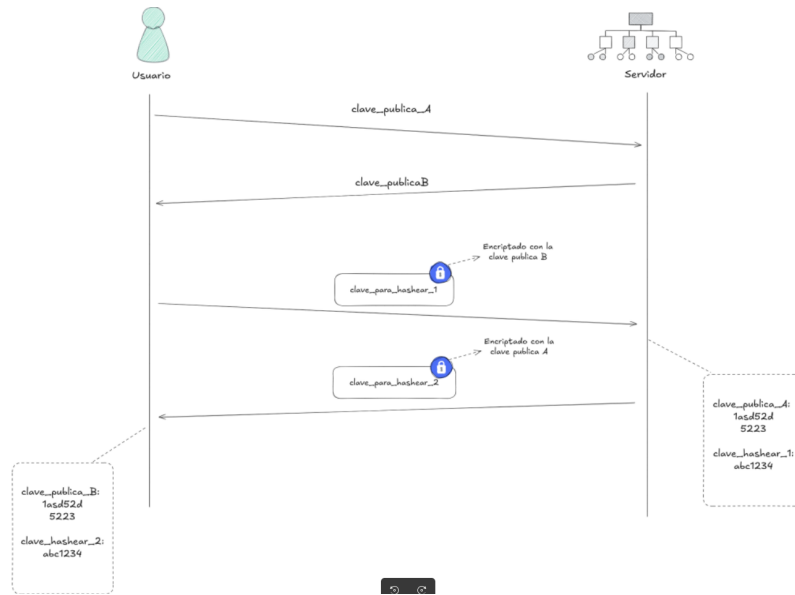


Figura 3: Encriptación con RSA y claves

3. Se adjunta la clave hash propia al final del mensaje, y luego se lo *hashea*.
4. Se compara el *hash* obtenido con el *hash* adjunto al final del mensaje: si coinciden, el mensaje es valido y el origen es confiable. De lo contrario, se descarta

El proceso recién descrito puede verse en la Figura 1.

Este metodo asegura la veracidad del origen del mensaje siempre, aprovechando que la clave enviada (la que se usa para hashear) solo podra ser descryptada y leída por una parte: quien genero y envio la clave publica inicialmente.

Uniendo estas dos ideas, los mensajes podrán enviarse encriptados, sin vulnerabilidades posibles (o al menos, no que se hayan detectado durante la realización de este trabajo). El flujo completo de mensajes para asegurar la encriptación de los mensajes puede verse en la Figura 2.

7. Servidor

En nuestro sistema de videoconferencias contamos con un servidor el cual juega un rol fundamental, ya que este actúa como **signaling server**.

Este facilita la comunicación entre los distintos usuarios que se encuentran dentro de la plataforma, permitiendo así que se conozcan para luego poder intercambiar la información relevante para establecer una conexión de tipo **Peer-to-Peer**(P2P) si así lo desearan.

7.1. Funcion del servidor

El servidor tiene como objetivo principal gestionar la comunicación entre usuarios dentro del programa.

7.1.1. Persistencia de usuarios

Para que un usuario que ya se registró no tenga que volver a hacerlo cada vez que se levanta el servidor se optó por hacer lo siguiente. Cuando se levanta el servidor se le envía un archivo `.conf` el cual posee la ruta de un archivo en el cual se persisten los usuarios de la manera `usuario;contraseña`. Luego cada vez que al servidor le llega una petición de registro la cual resulta exitosa, persiste esa información en ese mismo archivo.

7.1.2. Limitación de usuarios conectados

El servidor es quien permite o denega la solicitud de un usuario para conectarse a su plataforma. Un usuario conectado a este tiene la posibilidad de llamar o ser llamado, esto es posible una vez que el usuario inicia sesión en la plataforma. El servidor gestiona la cantidad máxima de usuarios conectados de manera simultánea en la plataforma. Si un usuario intenta loguearse pero ya se llegó a este tope se le notifica al usuario con el mensaje *"No hay lugar para una nueva conexión"* y no le permite ingresar a la plataforma dejándolo en la pantalla de lobby. Si luego de un tiempo el usuario vuelve a intentar ingresar y ya se desocupó algún lugar entonces podrá ingresar a la plataforma con normalidad.

7.1.3. Gestión de usuarios

Se mantiene de manera dinámica un registro de todos los usuarios registrados junto con sus estados en todo momento. El estado de un usuario puede variar entre:

- **Desconectado** : el usuario está presente en la plataforma, es decir que está registrado, pero no está conectado.
- **Disponible** : el usuario está conectado y disponible para realizar una videollamada con otro usuario.
- **Ocupado** : el usuario está conectado pero se encuentra realizando una videollamada con otro usuario.

Estos estados son gestionados y actualizados por el servidor. Quien también es el encargado de notificar a todos los usuarios cuyo estado sea *Disponible* sobre este cambio a actualizaciones.

7.1.4. Gestión interacción entre usuarios

El servidor es el responsable de coordinar el establecimiento de las conexiones entre los usuarios. Cuando un usuario decide llamar a otro el servidor facilita esta comunicación, siendo el quien se encarga de transportar la información necesaria para configurar la conexión P2P.

7.2. Estructura

A continuacion se detallara el diseño que se empleo para el servidor integrado tambien al protocoloPCA el cual ya fue desarrollado previamente.

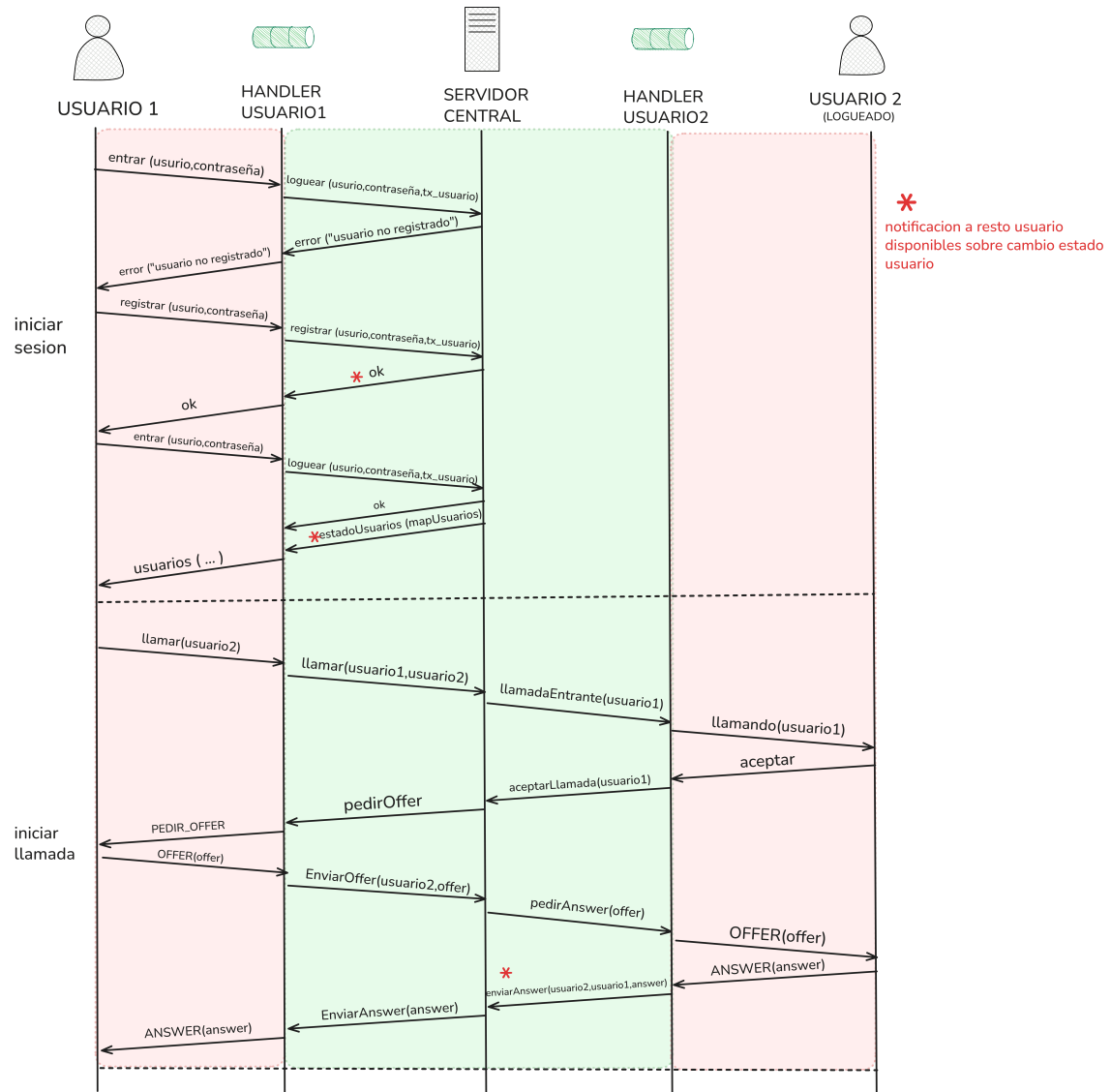


Figura 4: Protocolo de mensajes

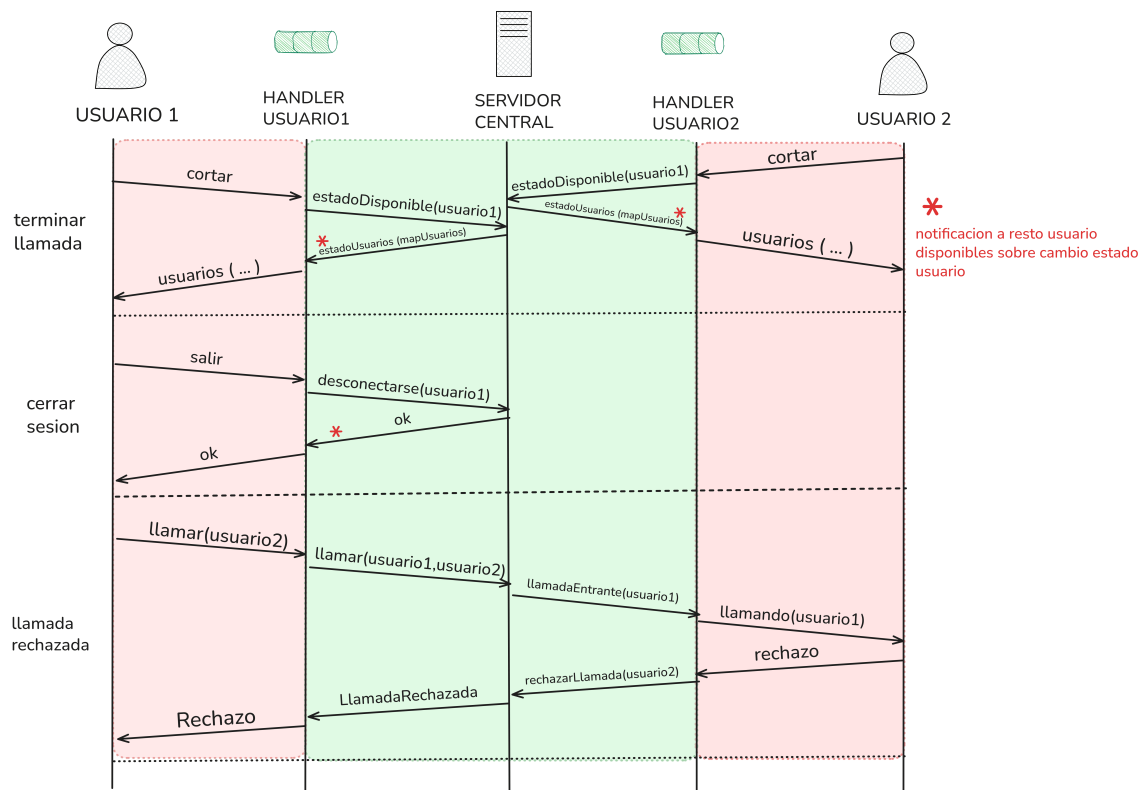


Figura 5: Protocolo de mensajes

Como se ve en los diagramas se opto por dividir la logica del servidor en dos partes:

- Logica del **Servidor Central**
- Logica de comunicación con el usuario por medio del protocoloPCA al cual se lo llamo **Handler Usuario**.

Con esto el Servidor Central se abstrae del protocoloPCA.

El flujo se basa en que hay una etapa de *logueo* en la cual el **Handler Usuario** le pedira al **Servidor Central** que o lo registre o lo logue con un nombre de usuario y ademas pasandole una punta de channel con el cual se comunicara una vez conectado.

Luego de esta etapa de logueo el servidor tendra al usuario como *conectado* y le enviara una notificacion cada vez que un usuario cambie su estado.

El servidor central y el handler usuario se comunican entre si por medio de channels de la manera en la que se ve en el diagrama.

El **servidor central** es un unico hilo el cual escucha las distintas peticiones que le llega de los handler usuario y las gestiona. Como se menciona anteriormente por cada usuario conectado tiene una punta del channel para comunicarse con este en caso de ser necesario.

Luego el **handler usuario** se divide en dos hilos de escucha, uno que se encarga de escuchar del *stream* lo que el usuario los pide y reedireccionarlo al servidor central y otro para escuchar las respuestas del **servidor central** y reedirigirlas al usuario.

8. Seguridad

8.1. DTLS

DTLS (Datagram Transport Layer Security) es un protocolo que tiene como objetivo asegurar el transporte de datos en aplicaciones de comunicación. Su filosofía de diseño tiene como base el construir TLS sobre UDP, ya que no es posible usar el protocolo de seguridad previamente mencionado sobre estos datagramas debido a que no tiene soporte ante casos de pérdida o reordenamiento de paquetes.

8.1.1. Crate UDP-DTLS

Para la implementación e incorporación de DTLS mostrada en el presente trabajo, utilizamos el crate `udp-dtls`, el cual presenta una abstracción en Rust de OpenSSL DTLS. Sus métodos internos facilitaron la creación y administración del contexto de sesión, permitiendo ejecutar el handshake completo y exportar el material criptográfico necesario para inicializar SRTP.

En particular, el crate ofrece primitivas para:

- asociar el contexto DTLS a sockets UDP ya existentes;
- procesar mensajes entrantes y generar las respuestas correspondientes del handshake;
- manejar retransmisiones y timeouts propios del protocolo;
- invocar la función `export_keying_material`, indispensable para obtener las claves maestras de SRTP.

A pesar de ofrecer una interfaz relativamente simple, la integración requirió adaptar su flujo de envío/recepción al modelo de threads del proyecto, ya que el crate no gestiona directamente el transporte.

8.1.2. Rol de DTLS - Flujo de aplicación

El protocolo DTLS ocupa un rol central dentro de la inicialización de una llamada segura. Su función principal en la aplicación es establecer un canal autenticado sobre un socket UDP del cual se derivará el material criptográfico a utilizar en SRTP para la generación de claves.

Dado su papel en la autenticación, se ubica entre la selección del candidato ICE y el comienzo de intercambio de paquetes mediante SRTP. Una vez finalizado el intercambio entre offerers y answerers y determinado el par de candidatos válidos, seleccionamos en la aplicación los sockets que utilizaremos para RTP y RTCP. Sobre ese socket se inicializa el handshake DTLS con los roles de cada peer determinados a partir del atributo `a = setup` en el SDP. Este rol define el orden de los mensajes del handshake, permitiendo una negociación estable entre ambos peers.

Antes de iniciar el handshake, cada peer ya ha generado su certificado X.509 autofirmado y ha publicado en el SDP su fingerprint SHA-256. Esta información es utilizada durante el intercambio DTLS para autenticar al peer remoto: cuando se recibe el certificado durante el handshake, su hash se compara con el fingerprint anunciado en el SDP. De este modo se valida la identidad del otro extremo.

Durante el handshake se intercambian, entre otros, los mensajes `ClientHello`, `ServerHello` y los certificados hasta llegar al estado `Finished`. Una vez finalizado esto, el protocolo proveerá la posibilidad de extraer el material criptográfico necesario para SRTP.

Es importante destacar que si el protocolo arroja un error en alguna de sus etapas, la comunicación no podrá continuar al no ser posible generar las claves SRTP necesarias para proteger la comunicación.

8.1.3. Generación de material criptográfico con OpenSSL

Para la generación del material criptográfico se tomó la decisión de utilizar el crate OpenSSL, el cual nos permite generar los elementos necesarios para la creación de diversos materiales necesarios para el handshake.

8.1.3.1. Generación y almacenamiento del certificado X.509

Ya que el protocolo y su contexto necesitan un certificado X.509, construimos el mismo utilizando la clave RSA (un algoritmo de clave pública utilizado en este tipo de certificados) para crear la clave privada propia del certificado. Una vez establecida la clave y el nombre correspondiente al certificado, necesitamos configurar las fechas de validez correspondientes al mismo junto a su firma.

Dado que `udp-dtls` trabaja con el tipo `Certificate`, realizamos la conversión correspondiente para posteriormente generar su `pkcs12`, un contenedor necesario para el `Identity` de `RTCPeerConnection` que almacena tanto el certificado como la clave privada.

8.1.3.2. Fingerprint SHA-256

Una vez que generamos el certificado X.509, es necesario obtener su huella digital derivada de la aplicación de la función hash SHA-256 sobre la representación DER binaria del certificado.

La huella digital cumple un rol fundamental dentro del SDP. Cada peer incluye en su offer o answer una línea `a=fingerprint:sha-256 <huella calculada>`, indicando aquella que corresponde al certificado que utilizará durante el handshake DTLS para validar su identidad.

Durante el handshake, cuando se recibe el certificado enviado por el peer remoto, la aplicación vuelve a calcular su hash de manera local y lo compara con la huella publicada previamente en el SDP. Si ambas coinciden, se autentica correctamente la identidad del peer y se valida el handshake.

8.1.4. Negociación de roles DTLS

8.1.4.1. Análisis del atributo setup en SDP

Como mencionamos en secciones anteriores, cada SDP correspondiente a un peer incluye la línea `a=setup:<rol>` de la cual determinaremos si el peer toma el papel de cliente o de servidor en la comunicación.

En la implementación realizada, consideramos que el offerer siempre comenzará con `a=setup:actpass` dado que aún no sabe si va a actuar como cliente o servidor DTLS al depender esa decisión del setup que adopte el answerer. Al recibir una offer con `actpass`, el answerer tiene la posibilidad de responder con `active` o `passive`, indicando si tomará el rol de cliente e iniciará por sí mismo el `ClientHello` o esperará a que lo haga el offerer.

Tomando en cuenta la convención utilizada en WebRTC general y sus diversas implementaciones, decidimos que el answerer responda por defecto con `a=setup:active`. Se menciona en el estándar relacionado a este tópico que el adoptar esta elección se reduce la latencia, ya que el answerer inicia el handshake de forma inmediata sobre el par de candidatos ICE previamente seleccionado.

8.1.4.2. Determinación de rol local

Una vez analizado el atributo `a=setup` del SDP remoto como se menciona en la sección anterior, se procede a determinar el rol local DTLS de cada peer.

La determinación no depende únicamente del valor del atributo `a=setup`, sino también de si el peer local es offerer o answerer.

La lógica implementada en la aplicación sigue esta convención:

- Si somos **offerer**: interpretamos el valor remoto directamente.
 - remoto = **active** → local = **passive** (Servidor DTLS)
 - remoto = **passive** → local = **active** (Cliente DTLS)
 - remoto = **actpass** → local = **passive** (Servidor DTLS)
- Si somos **answerer**: el offerer siempre envía **actpass**, y como explicamos anteriormente adoptamos el rol **active** (Cliente DTLS).

8.1.5. Handshake DTLS - Flujo

El flujo del handshake comienza una vez determinado el rol local a partir del SDP. En el caso del rol **active**, el peer local inicia la negociación ejecutando `DtlsConnector::connect()`, mientras que si el rol es **passive**, el peer emplea `DtlsAcceptor::accept()` y aguarda el `ClientHello` generado por el otro extremo.

La secuencia de mensajes del handshake en su entera es gestionada por la propia implementación de OpenSSL utilizada a través del crate `udp-dtls`. El crate mantiene internamente los estados del protocolo, maneja retransmisiones ante pérdida de datagramas, fragmentación, reensamblado y autenticación de cada mensaje sin intervención de la aplicación.

La aplicación no procesa ni entrega explícitamente datagramas DTLS. En su lugar, la comunicación se realiza mediante un `UdpChannel` que une el socket con la dirección remota. Este canal permite que `connect()` o `accept()` lean y envíen datagramas DTLS directamente a través del socket. Durante este proceso, el contexto DTLS valida el certificado del peer remoto comparando su huella digital con el fingerprint anunciado previamente en el SDP como se explicó en secciones anteriores.

Una vez que ambos peers intercambian los mensajes **Finished**, el handshake concluye y el canal DTLS queda autenticado, habilitando la extracción del material criptográfico necesario para SRTP.

8.1.6. Extracción de claves SRTP

8.1.6.1. Derivación de claves

Una vez completado el proceso de handshake DTLS, estamos en condiciones de derivar las claves para SRTP. Para realizar la derivación utilizamos el crate `OpenSSL` y su método `export_keying_material`, el cual nos permite trabajar con material criptográfico a partir del estado del handshake DTLS y nos devuelve un buffer con la siguiente estructura definida en estándares RFC

- `client_write_key`
- `server_write_key`
- `client_write_salt`
- `server_write_salt`

Luego, a partir de el rol del peer, determinaremos que items de la estructura utilizaremos para enviar y cuales para recibir. Si el peer actúa como cliente, utilizará la clave y el salt del cliente para transmitir y los del servidor para recibir, mientras que si actúa como servidor invertirá dicha asignación.

8.1.6.2. Estructura de claves

Como puede observarse en la lista perteneciente al ítem anterior, podemos separar el material de claves exportado de DTLS en los siguientes dos sub-ítems:

- **Key:** Es la clave base a partir de la cual el protocolo SRTP genera el stream utilizado para cifrar e interpretar campos del paquete RTP. No se usa directamente sino que se combina con valores dependientes del paquete (como por ejemplo el ROC) para derivar claves de sesión de forma específica para cada paquete.
- **Salt:** Es un valor aleatorio que actúa como campo adicional en la derivación interna de SRTP. Su principal rol es asegurar la variabilidad en la generación de keystreams (que nos ayudarán en el cifrado) al derivar el IV (vector de inicialización) y utilizar contadores internos del protocolo (SSRC, ROC, etc) que pueden llegar a seguir un patrón predecible. Combinando el salt con los campos previamente listados construirá un IV único para cada paquete. En secciones posteriores se desarrollará con mayor profundidad el proceso de cifrado.

8.1.6.3. Preparación del contexto SRTP

Una vez derivadas las claves y salts desde DTLS, la aplicación inicializa dos instancias independientes de **SRTPContexto**: una para transmisión (TX) y otra para recepción (RX). Esta separación permite aplicar correctamente las claves derivadas según el rol DTLS.

Cada contexto almacena:

- la clave y salt asignados para su dirección;
- el contador ROC (rollover counter), inicializado en cero;
- la secuencia RTP más alta recibida;
- la ventana anti-replay utilizada por SRTP.

En esta fase no se realiza cifrado sino que simplemente se deja preparado el estado interno necesario para que los hilos de RTP puedan invocar las funciones de protección o verificación a medida que se envían y reciben paquetes.

8.2. SRTP

8.2.1. Rol de SRTP - Flujo de aplicación

SRTP es el protocolo encargado de proteger los paquetes RTP una vez finalizado el handshake DTLS. Su rol en la aplicación es:

1. Proteger la confidencialidad del payload RTP mediante cifrado.
2. Garantizar autenticidad e integridad mediante un tag HMAC-SHA1-80 que se adjunta a cada paquete.

Una vez que DTLS concluye y se exportan las claves, SRTP opera de manera independiente.

8.2.2. Arquitectura del módulo

El módulo de SRTP se estructura en torno al struct **SRTPContexto**, que se encarga de encapsular todo el estado necesario para aplicar el perfil SRTP negociado durante DTLS.

Cada contexto contiene los siguientes elementos:

- **clave SRTP**
- **salt SRTP**
- **ROC (Rollover counter)**

- **seq_mas_alto** (Secuencia RTP más alta observada)
- **Replay window**

Cada peer contiene dos contextos independientes, uno de transmisión y otro de recepción, almacenados en el struct `RTCPeerConnection`.

8.2.2.1. Contextos de cifrado y decifrado

Dado que encapsulamos toda la lógica del contexto SRTP en el struct detallado anteriormente y como esto se maneja en cada peer, cada contexto puede actualizar su ROC, trackear su `seq_mas_alto` y operar la replay window.

El módulo de contexto es responsable de:

- Reconstruir el número de secuencia extendido
- Actualizar el ROC cuando la secuencia vuelve a cero
- Prevenir ataques de replay verificando si un paquete ya fue aceptado
- Generar el IV de cifrado a partir de clave, salt y ESN
- Cifrar/decifrar usando AES-CTR
- Verificar autenticidad

8.2.2.2. Integración con threads RTP existentes

Los contextos SRTP se integran de forma directa en los hilos ya existentes de la aplicación de la siguiente manera:

- **En el thread de envío RTP:**
 1. se construye el paquete RTP de forma habitual,
 2. se pasa a la función **proteger_y_firmar_rtp**,
 3. se genera el IV usando la clave de transmisión, salt, SSRC y número de secuencia extendido (formado por seq + ROC),
 4. se cifra el payload,
 5. se genera el tag HMAC-SHA1-80,
 6. se envía el paquete protegido por el socket.
- **En el thread de recepción de paquetes, cada paquete recibido se procesa de la siguiente forma:**
 1. Se extrae seq, SSRC y payload,
 2. Se llama a la función **verificar_y_desproteger_rtp** que:
 - calcula el índice de paquete (seq extendida + ROC),
 - aplica la lógica anti-replay,
 - vuelve a calcular el tag de autenticación y lo compara,
 - descifra el payload in-place,
 - y finalmente trunca el vector para remover el tag SRTP y dejar sólo RTP.
 3. Con el **paquete_srtp** ya convertido nuevamente en RTP, se llama a `PaqueteRTP::try_from` y el payload se envía al jitter buffer.

De esta forma, SRTP queda encapsulado entre la construcción/parsing de `PaqueteRTP` y el uso de `SocketUDP`, sin modificar el resto de la lógica de transmisión de video.

8.2.3. Claves y perfiles

8.2.3.1. Uso de las claves derivadas por DTLS

Una vez ejecutado el handshake DTLS y validado el certificado remoto, el método `export_keying_material` de OpenSSL nos proporciona un bloque de bytes con todas las claves necesarias para SRTP. La estructura obtenida incluye las claves y salts de cliente y servidor en el orden definido por el perfil SRTP negociado.

Como se mencionó en secciones anteriores, en nuestra implementación, la estructura `ClavesSRTP` ya devuelve las claves en el orden correcto (TX y RX) según el rol DTLS local. De esta manera, la función `crear_contextos_srtp` simplemente instancia:

- `contexto_srtp_tx` con `clave_tx` y `salt_tx`,
- `contexto_srtp_rx` con `clave_rx` y `salt_rx`.

Estos vectores quedan almacenados en la estructura `RTCPeerConnection` y son utilizados en los threads RTP. No existe intercambio adicional de claves después del handshake: SRTP funciona de forma completamente local en cada peer.

8.2.3.2. Perfil SRTP

Durante el armado del `DtlsConnector`, se decide utilizar el perfil `AES_128_CM_SHA1_80` dentro de nuestra implementación al ser el más usado en WebRTC y permitir un control más preciso en cada etapa del proceso de cifrado que otros perfiles. En este perfil:

- el cifrado es AES-128 en modo CTR,
- la autenticación se realiza mediante HMAC-SHA1,
- se utiliza un tag truncado de 80 bits (10 bytes),
- los salts son de 112 bits (14 bytes),
- el IV se deriva combinando salt, SSRC y el índice de paquete.

8.2.3.3. Salt y master keys

En SRTP, la clave y el salt cumplen roles distintos. La **clave** provee el material secreto para generar el keystream AES-CTR, mientras que el **salt**, como se mencionó anteriormente, se utiliza para construir IVs únicos para cada paquete.

En cada llamada a `derivar_iv`, nuestra implementación:

- inicializa un arreglo de 16 bytes,
- copia los 14 bytes del salt en las posiciones `[2..16)`,
- aplica XOR con el SSRC del flujo,
- aplica XOR con el índice extendido del paquete (`ROC + seq`),

Esto garantiza que cada paquete reciba un IV distinto, mayormente gracias al salt. De esta forma evitamos vulnerabilidades causadas por la reutilización de un keystream.

8.2.4. Cifrado de RTP

8.2.4.1. Comparación de estructura de paquete RTP

Nuestra implementación conserva por completo el formato estándar del header RTP presentado en la entrega intermedia dado que SRTP actúa solamente sobre:

- el payload del paquete
- el tag final de autenticación

8.2.4.2. Proceso de cifrado

El flujo de cifrado implementado en `proteger_y_firmar_rtp` funciona así:

1. Se extrae el número de secuencia del header.
2. Se calcula el índice extendido del paquete con `calcular_indice_paquete`.
3. Se deriva el IV mediante `derivar_iv`.
4. Se cifra el payload con AES-CTR usando `proteger_rtp`.
5. Se genera un tag de autenticación HMAC-SHA1-80 con `generar_tag_autenticacion`.
6. Se adjuntan los 10 bytes del tag al final del paquete.

Este proceso no altera el header ni la longitud del payload, salvo por los 10 bytes adicionales del tag, requisito estándar del perfil SRTP.

8.2.4.3. Replay y autenticación - ROC

El mecanismo anti-replay está implementado completamente dentro de `calcular_indice_paquete` y `verificar_replay_y_actualizar`.

En nuestra implementación:

- **ROC** se incrementa únicamente cuando detectamos que la secuencia salta hacia atrás de manera suficientemente grande como para interpretarse como rollover (más de la mitad del espacio de 16 bits).
- **seq_mas_alto** se actualiza solamente cuando recibimos un número de secuencia estrictamente mayor al conocido, evitando interpretar reordenamientos como rollover.
- La **replay window** de 64 bits permite aceptar paquetes reordenados, pero rechaza los que ya fueron vistos o que son demasiado viejos.

8.2.5. Decifrado de RTP

8.2.5.1. Validaciones

Antes del descifrado, `verificar_y_desproteger_rtp` realiza:

- verificación de longitud mínima (RTP + tag),
- separación del tag recibido,
- reconstrucción del índice extendido,

- verificación anti-replay,
- comparación del tag HMAC-SHA1-80 con el recalculado.

Cualquier error detiene el procesamiento y el paquete se descarta sin llegar al decodificador de video.

8.2.5.2. Decodificación

Superadas las validaciones anteriores, el payload se descifra con `descifrar_rtp`, que vuelve a generar el IV original y ejecuta AES-CTR en modo **Decrypt**. El header permanece intacto y el paquete vuelve a su forma RTP estándar al truncar el paquete y remover el tag.

8.2.5.3. Verificación de autenticidad

La autenticidad se verifica recalculando el HMAC-SHA1-80 con `generar_tag_autenticacion` y comparando. Si pasó la autenticación y el anti-replay, procedemos a descifrar el payload in-place.

9. Conclusión

El desarrollo de RoomRTC no solo implicó implementar los componentes fundamentales del ecosistema WebRTC, sino también atravesar un proceso de aprendizaje profundo como equipo de trabajo. A lo largo del proyecto adquirimos experiencia práctica en la coordinación de tareas, el diseño incremental de software y la resolución conjunta de problemas complejos, especialmente en áreas como protocolos de red y seguridad.

Trabajar con tecnologías de bajo nivel, como DTLS, SRTP y RTP, nos obligó a comprender en detalle el flujo completo de una llamada en tiempo real, desde la negociación inicial hasta la transmisión segura de video. Esto reforzó la importancia de la modularización, el testing y la documentación clara para sostener un proyecto de esta escala.

Asimismo, la necesidad de integrar múltiples componentes, como lo son la interfaz gráfica, servidor central, negociación ICE, transmisión de video y manejo de estados, nos permitió ejercitar habilidades de diseño, comunicación interna y priorización del trabajo. El enfoque iterativo con entregas semanales también nos llevó a mejorar nuestra capacidad de planificación, revisión de código y adaptación ante cambios de requerimientos.

10. Anexo - Captura y transmisión de audio

Una vez establecida la sesión RTP entre dos clientes, estos tienen la capacidad de enviar audio o video. En esta sección explicaremos como se logra la captura del audio, y como se integro la transmisión de este a la sesión RTP para poder ser recibido y transmitido por el otro cliente.

10.1. Captura del audio

Para la captura del audio, el objetivo planteado fue permitir que se envíe audio durante la llamada, dándole al usuario la posibilidad de silenciar el micrófono siempre que lo requiera. Esto exigió que el componente Micrófono deba ser necesariamente un componente multi-hilo, ya que mientras el audio se transmite el usuario debe poder enviar un mensaje que derive en que al micrófono se le solicite que detenga su reproducción. A continuación, se explicara cuales fueron esos hilos y como se integro esto con el funcionamiento del *crate* utilizado para capturar el audio.

Para la captura del audio se utilizo el *crate* CPAL, que permite tanto capturar audio como reproducirlo. La captura de audio funciona con un hilo creado por el mismo *crate*. Ese hilo ejecuta un *callback* creado por nosotros cada vez que se termino de capturar un *buffer* de audio de un tamaño determinado. Además, luego de crearse ese hilo, el *crate* nos da acceso a un 'objeto' que permite controlar la ejecución de ese hilo, permitiéndonos pausar o des-pausar la captura.

Una vez explicado el funcionamiento del *crate* utilizado, podemos detallar nuestra implementación. Para lograr la funcionalidad requerida, el Micrófono administra dos hilos. El primero hilo del micrófono es aquel que captura el audio, que es creado por el *crate* y que luego podemos administrar para solicitar que pause o des-pause la captura. Cada vez que se ejecute el *callback* en ese hilo, se reenviaran las muestras de audio al segundo hilo del micrófono: el hilo de retransmisión. El hilo de retransmisión cumple principalmente dos funciones: retransmitir el audio proveniente del hilo de captura, y reemplazar el receptor del audio. La capacidad de cambiar quien sera el receptor del audio facilita el uso de Micrófono, sobre todo a la hora de realizar múltiples llamadas: si se corta una llamada y se inicia una nueva, se podrá seguir usando el mismo micrófono si se le indica a este el nuevo receptor para el audio. En ese caso, el Micrófono enviara una instrucción al hilo de retransmisión indicándole que hay un nuevo receptor para el audio. Así, como el micrófono siempre es el mismo, Llamada podrá tener una referencia al micrófono, y cuando el usuario desea silenciar el audio lo único que deberá hacer sera delegar esa tarea. En la Figura 6 se puede observar la comunicación entre los diferentes hilos para capturar el audio y retransmitirlo al receptor.

El diseño elegido para el 'objeto' Micrófono permite lograr uno de los objetivos perseguidos durante el desarrollo de este trabajo, que consiste en que los componentes tengan el mismo tiempo de vida que la aplicación. Al tratarse de componentes multi-hilo, la creación y destrucción constante de estos para cada llamada diferente ocasionaba problemas difíciles de resolver. Se debía prestar especial atención a que los hilos se cierran y vuelvan a abrir correctamente, evitando cualquier *race condition* posible durante este proceso. Además, se debía tener en todo momento la información necesaria para volver a crear ese componente, lo cual ocasionaba que otras partes de la aplicación estén fuertemente acopladas a los colaboradores internos del mismo. Estas razones favorecieron la búsqueda de que los componentes estén pensados para poder reutilizarse entre llamadas: una vez creado el componente, se busco tener las herramientas necesarias para poder usarlo durante todo el flujo de la aplicación, sin necesidad de reemplazarlo. Vale aclarar que las estructuras fueron pensadas para que esto no suponga un coste de rendimiento, y por ello el diseño también contempla que los hilos de estos componentes deben estar bloqueados mientras el componente este inactivo, ocupando un mínimo de recursos. Con esto en mente, resulta natural el surgimiento del hilo de retransmisión para el micrófono, ya que este actúa como una herramienta cuya función es permitir que el audio capturado pueda dejar de retransmitirse a una llamada anterior, para comenzar a retransmitirse a una nueva llamada. La Figura 7 muestra como se logra que el microfono pueda reutilizarse entre diferentes llamadas.

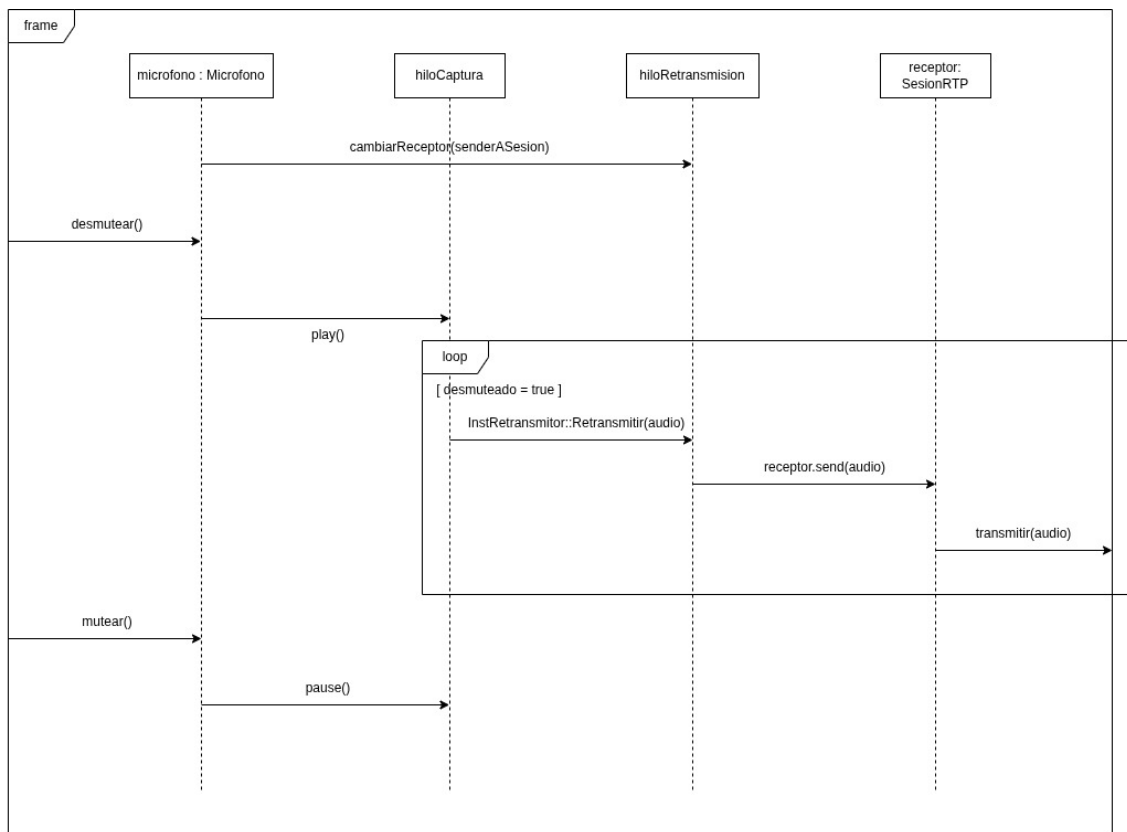


Figura 6: Diagrama de secuencia - Captura del Microfono

La Figura 8 corresponde a un diagrama de actividad *UML* que representa el funcionamiento explicado. Se eligió este tipo de diagramas ya que dentro de los diagramas especificados por *UML* es el único que permite explicitar la creación de nuevos hilos de ejecución. Cabe aclarar que las condiciones *esIniciarLlamada* y *esDesmutear* no existen en la implementación realmente, sino que se utilizaron para representar cual evento es el que produce cada flujo dentro de la aplicación.

10.2. Transmision de audio

Para transmitir el audio capturado del cliente se utilizo la misma sesion RTP cuyo funcionamiento se explico en la sección anterior. El protocolo *RTP* indica que se pueden mantener multiples flujos de contenido multimedia, siempre que se indique claramente el tipo de paquete o *Payload Type*, adjuntando un identificador *SSRC* único a los paquetes de cada flujo. En nuestra implementación, se siguió con esa idea, que permitio que los cambios necesarios para transmitir el audio sean minimos. Cuando el Cliente B recibe un paquete enviado por el Cliente A, puede comprobar si es audio o video revisando el *Payload Type* del paquete, y delegar la reproducción al reproductor de video o al reproductor de audio según sea adecuado.

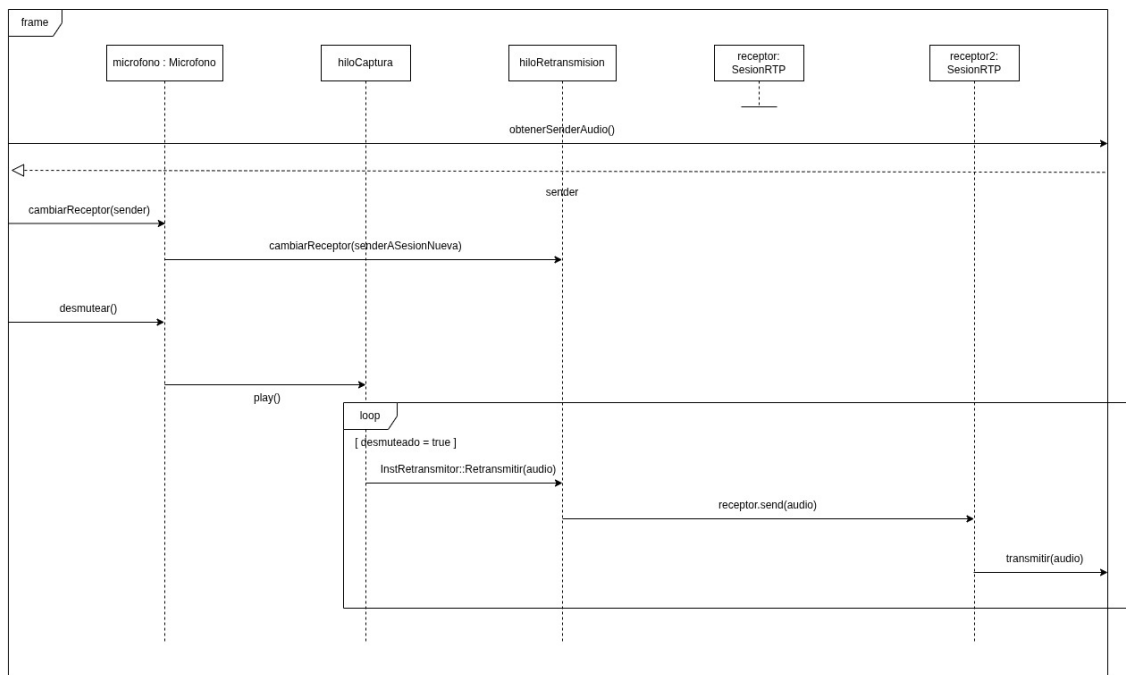


Figura 7: Diagrama de secuencia - Cambio de receptor del microfono

11. Anexo II - Envío y recepción de archivos

Dentro de la llamada, contemplando modelos vigentes de aplicaciones de videollamadas y ateniéndonos a la consigna brindada por la cátedra, ambos peers deben poder enviarse archivos en el transcurso de su comunicación y decidir si quieren o no aceptar y descargar lo ofertado por el peer emisor. A lo largo de esta sección, desarrollaremos los protocolos que nos permitieron llegar a esto, asimismo se planteará el demultiplexado requerido por WebRTC y la arquitectura multi-threading que debimos expandir para lograr la emisión y recepción correcta de este evento.

11.1. Protocolos

Para desarrollar este feature debimos implementar, con ayuda de crates, los protocolos **SCTP** (Stream Control Transmission Protocol) y **DCEP** (Data Channel Establishment Protocol) para así poder establecer la comunicación necesaria

11.1.1. SCTP

Para la implementación del protocolo SCTP, montado sobre DTLS, se utilizó el crate *sctp_proto* con el fin de manejar asuntos relacionados a la lógica interna del protocolo. Dicho crate nos permitió trabajar con el concepto de **Endpoint**, que maneja las asociaciones por socket y eventos relacionados, y **Asociaciones** encargadas de contener múltiples streams y ocuparse de los paquetes tanto entrantes como salientes.

11.1.1.1. Loop de procesamiento y sincronización En el diseño propuesto, el procesamiento de SCTP es realizado casi en su entera en un loop dedicado que trabaja en un thread separado. Dicho bucle se encarga de:

- Recibir y leer datagramas entrantes encapsulados en DTLS.

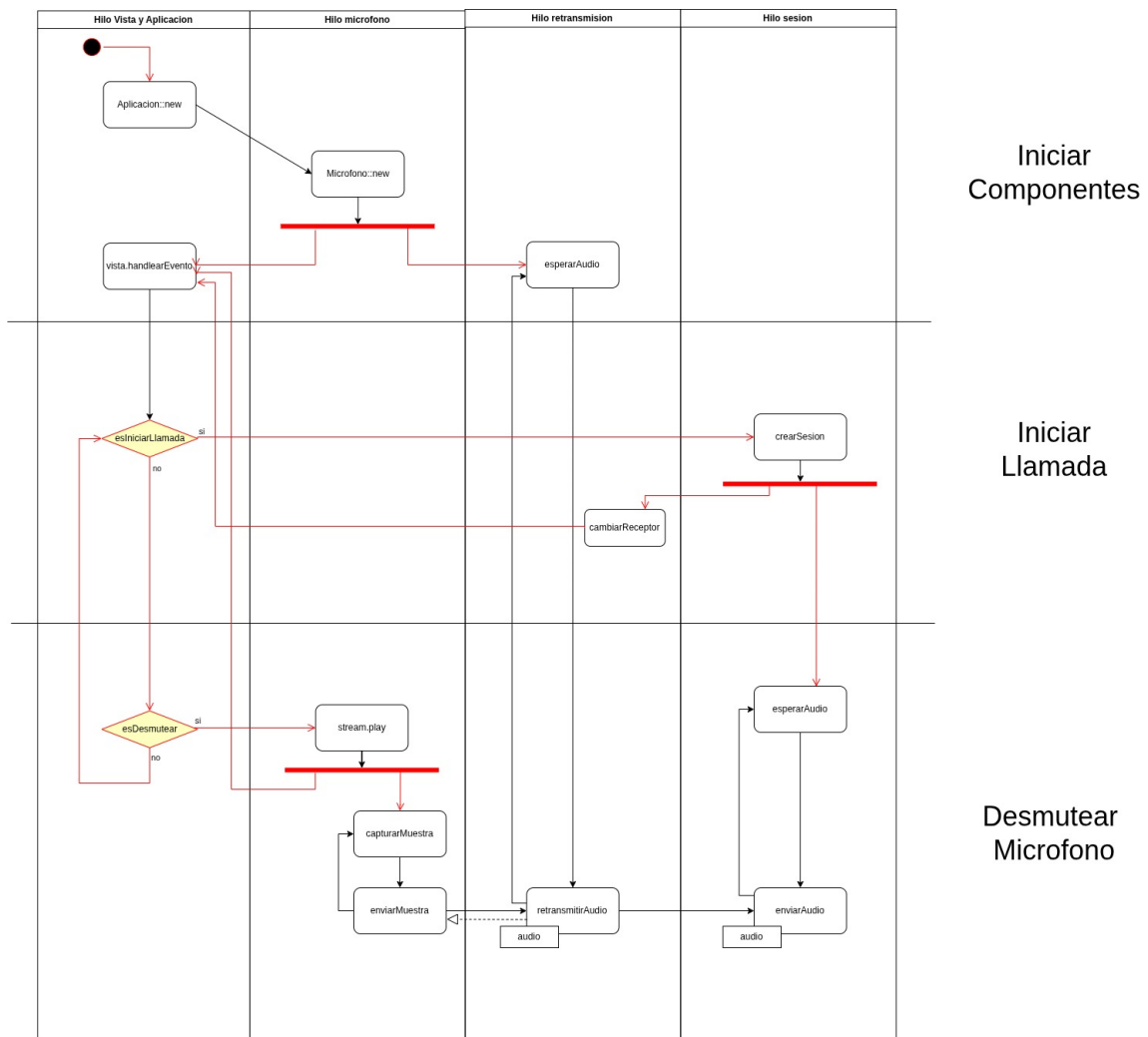


Figura 8: Diagrama de actividad de la captura de audio

- Manejar eventos de aplicación, incluyendo la apertura del DataChannel si esta es solicitada.
- Enviar datos pendientes en caso de ser necesario.
- Sincronizar el endpoint con la asociación.
- Drenar y enviar las transmisiones necesarias por DTLS.
- En caso de que el peer ordene cerrar la conexión, se encarga además de enviar el `shutdown` correspondiente, cerrando así la asociación activa y limpiando el handle.

11.1.2. DCEP

El rol principal de **DCEP** en el envío y recepción de archivos es el de establecer las características principales de un canal (tales como el tipo de canal, protocolo, prioridad y confiabilidad del mismo) y procesar su apertura de forma ordenada. Para lograr este protocolo se trabajó con dos módulos encargados de lo siguiente:

- `dcep_handshake.rs`: Como su nombre lo indica, se encarga de la implementación del **Handshake** DCEP para establecer un data channel a través de SCTP, incluyendo tanto `DATA_CHANNEL_OPEN` como `DATA_CHANNEL_ACK`.

- **data_channel_open_serde.rs**: Se encarga de la serialización y deserialización de los mensajes de apertura de canal. Incluye información sobre el tipo de canal, prioridad, parámetros de confiabilidad, etiqueta y protocolo.

Considero importante nombrar además el módulo **protocolo_archivo.rs**, donde se establece el enum **MensajeArchivo** utilizado a lo largo del programa (junto a **EventoSCTP**) para el manejo de eventos relacionados a la negociación de archivos. Dentro se definen los siguientes mensajes:

- **OfertaArchivo**: Se envía cuando Peer A quiere enviarle un archivo a Peer B, se considera una "oferta" al consultarle al peer receptor si está de acuerdo con recibir y guardar el disco el archivo ofertado.
- **AceptarArchivo**: Se envía cuando Peer B acepta la oferta de Peer A para informarle al peer emisor que puede comenzar con el envío del archivo por el stream en chunks. Se decidió no enviar el archivo si este no es aceptado para no desperdiciar recursos.
- **RechazarArchivo**: Se envía cuando el peer receptor rechaza la oferta; no ocurre ningún envío ni recepción.
- **DatosArchivo**: Contiene los datos (ya sea fragmentados o en su totalidad, dependiendo el tamaño) del archivo.
- **FinArchivo**: Lo utilizamos para informar del fin de la transferencia. Es útil al haber establecido un procesamiento en chunks, de esta forma informamos cuando estamos habilitados a guardarlo en el disco del peer receptor.

11.2. Demultiplexado

Se tomó la decisión de **demultiplexar** basándonos en la arquitectura real de WebRTC. Quién anteriormente era únicamente nuestro socket dedicado a SRTP post-ICE y post-DTLS (cuando el protocolo era utilizado únicamente para negociar keys) ahora contendrá un demultiplexador que, basado en una lógica explicada en la siguiente sección, redirigirá paquetes **SRTP**, **DTLS** y **SCTP/DCEP** según corresponda; este mecanismo se basa, según el estándar, en la idea de tener todo lo necesario en un único lugar y a partir de ello comunicarlo donde se lo necesite.

11.2.1. Identificación y separación

Para una correcta identificación de paquetes, se tomaron en cuenta determinados patrones de bytes con el fin de determinar el tipo de paquete entrante y redirigirlo a su channel correspondiente. Los criterios adoptados fueron los siguientes:

- **DTLS**: se identifica por el *ContentType* del registro DTLS en el primer byte. En particular, DTLS usa valores en el rango `0x14..0x17` (20..23), correspondientes a *ChangeCipherSpec*, *Alert*, *Handshake* y *ApplicationData*. Por lo tanto, si `paquete[0] ∈ [20,23]` se reenvía al canal `dtls_tx`.
- **SRTP (RTP)**: se identifica verificando que el paquete tenga al menos 2 bytes y que los dos bits más significativos del primer byte indiquen versión 2 de RTP (`paquete[0] & 0xC0 == 0x80`). Para evitar confundirlo con RTCP, se usa además el segundo byte como filtro: si `paquete[1] < 192` se considera RTP y se reenvía al canal `srtp_tx`.
- **RTCP y STUN**: para STUN se utiliza su firma característica (cabecera con los dos bits más altos en cero y *magic cookie* `0x2112A442` en los bytes 4..7). Estos paquetes se ignoran ya que corresponden a señalización/chequeos y no forman parte del flujo SRTP/DTLS una vez establecida la comunicación. En cuanto a RTCP, sus tipos suelen ubicarse en `[192,223]`, por lo que quedan excluidos del criterio anterior de RTP; si se desea procesarlos, puede agregarse una detección específica y reenviarlos a un canal dedicado (o al mismo canal de SRTP según el diseño).

11.3. Arquitectura multithread

11.3.1. Multi-threading

La incorporación del envío y recepción de archivos implicó extender la arquitectura multithread existente. Dado que la transferencia de archivos puede involucrar operaciones costosas (lectura/escritura en disco y envío en chunks), se decidió aislar algunas de estas tareas en hilos dedicados, evitando que interfieran con los flujos críticos de audio y video en tiempo real. En los diagramas se obvia la existencia del thread general de SRTP/RTCP, adicionalmente, se informa que el demultiplexador corre en un thread aparte.

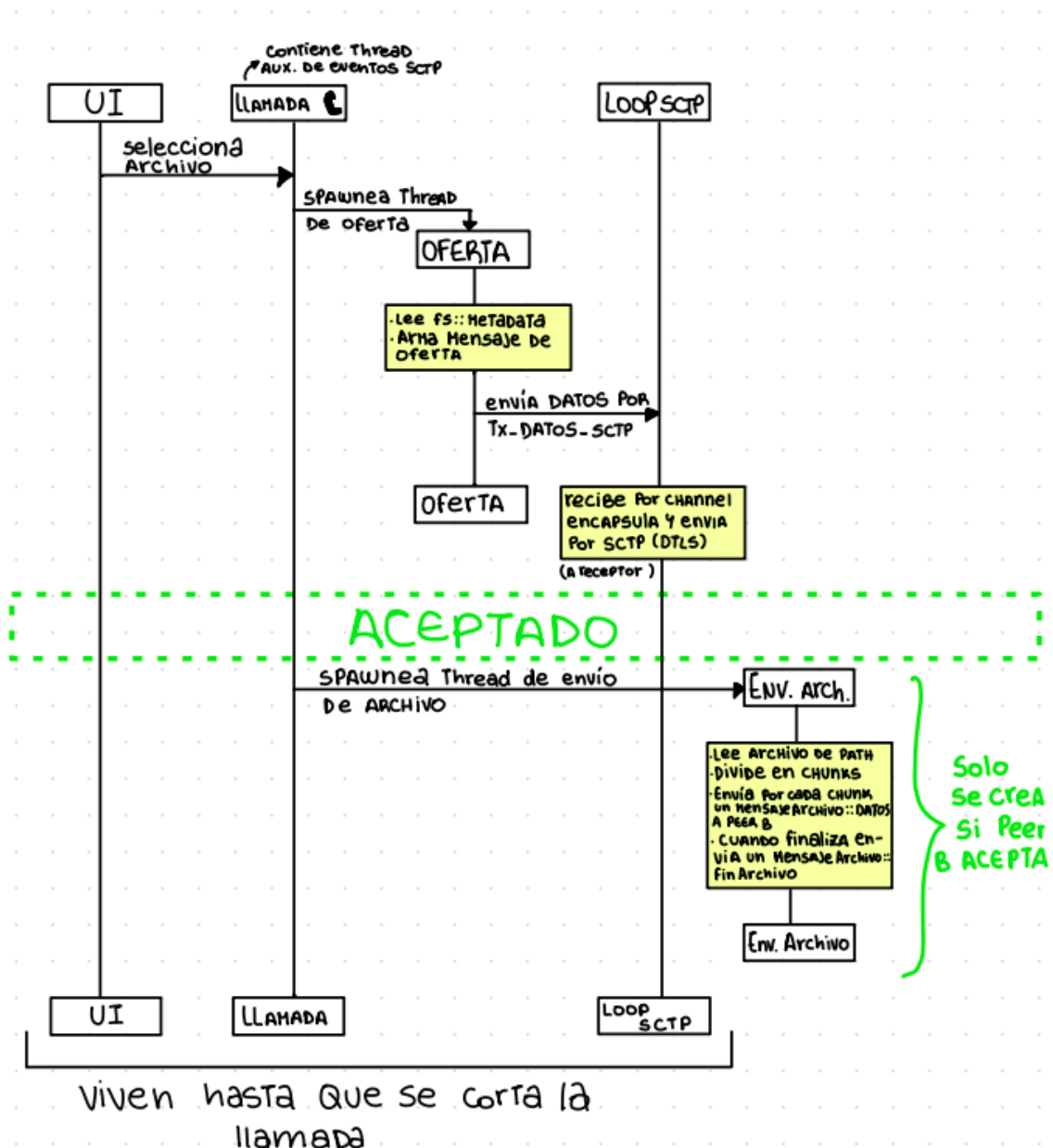


Figura 9: Diagrama de actividad de peer emisor al enviar archivo

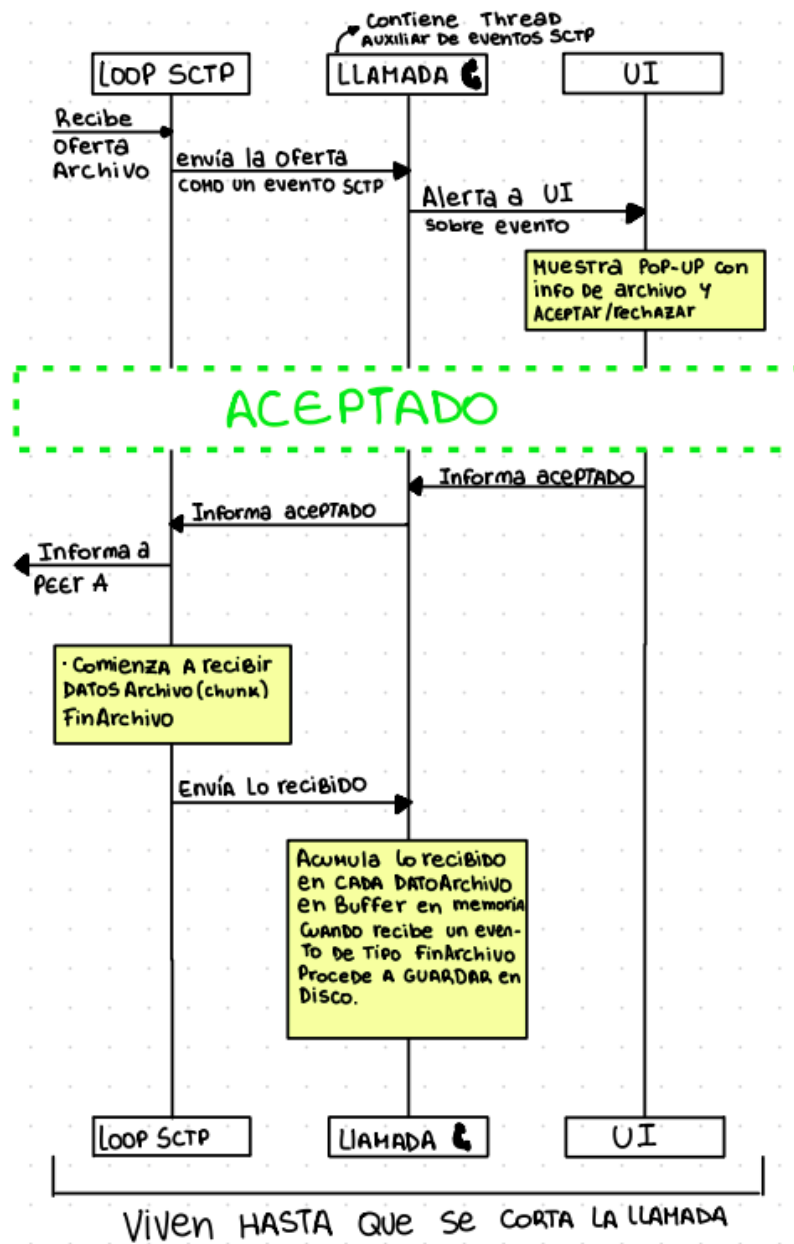


Figura 10: Diagrama de actividad de peer receptor al recibir archivo

Referencias

Deacon, John (2009). «Model-view-controller (mvc) architecture». En: *Online*[Citado em: 10 de março de 2006.] <http://www.jdl.co.uk/briefings/MVC.pdf> 28, pág. 61.